

R4DS 12 – Advanced Data Import

Spreadsheets

Reading Excel Files

`readxl::read_excel()` reads both `.xls` and `.xlsx`:

```
library(readxl)
students <- read_excel("students.xlsx",
  col_names = c("id", "name", "food",
                "meal_plan", "age"),
  skip = 1,
  na = c("", "N/A"),
  col_types = c("numeric", "text", "text",
                "text", "text"))
```

Key arguments:

Argument	Purpose
<code>col_names</code>	Override column names
<code>skip</code>	Skip header rows
<code>na</code>	Strings to treat as NA
<code>col_types</code>	Force column types
<code>range</code>	Read specific cells ("A5:F15")

Multiple Sheets

`excel_sheets()` lists all sheets; `sheet` argument selects one:

```
excel_sheets("penguins.xlsx")

# Read and combine all sheets
penguins <- bind_rows(
  read_excel("penguins.xlsx",
    sheet = "Torgersen Island", na = "NA"),
  read_excel("penguins.xlsx",
    sheet = "Biscoe Island", na = "NA"),
  read_excel("penguins.xlsx",
    sheet = "Dream Island", na = "NA"))
```

Write Excel with `writexl::write_xlsx()`:

```
library(writexl)
bake_sale <- tibble(
  item = c("brownie", "cupcake", "cookie"),
  quantity = c(10, 5, 8))
write_xlsx(bake_sale, path = "bake-sale.xlsx")
```

Google Sheets with `googlesheets4`:

```
library(googlesheets4)
gs4_deauth() # for public sheets
read_sheet("sheet_url_or_id",
  col_types = "dcccc") # shorthand types
```

- 1 Read a messy Excel file and clean: handle merged cells with `fill()`, filter out non-data rows.
- 2 What does `janitor::clean_names()` do? How does it improve imported column names?
- 3 What happens if you use `read_xls()` on an `.xlsx` file?

Solutions (Spreadsheets)

- 1 Read with `read_excel()`, then pipe to `fill()` for merged cells and `filter(!is.na(...))` to remove blank rows.
- 2 `clean_names()` converts to snake_case, removes special characters, and handles duplicates:

```
janitor::make_clean_names(  
  c("First Name", "% Change", "1st Column"))
```

```
#> [1] "first_name"      "percent_change" "x1st_column"
```

- 3 It will error or produce garbled results — the formats are different (`.xls` is binary, `.xlsx` is XML-based).

Databases

Connecting with DBI

```
library(DBI)
library(duckdb)

# Connect (duckdb runs in-process, no server)
con <- dbConnect(duckdb())
dbWriteTable(con, "mpg", ggplot2::mpg)
dbWriteTable(con, "diamonds", ggplot2::diamonds)

# List tables and read
dbListTables(con)
dbReadTable(con, "mpg") |> as_tibble()
```

Other backends: `RPostgres`, `RMariaDB`, `RSQLite`, `odbc`.

Lazy Queries with dbplyr

`tbl()` creates a **lazy** reference — no data is fetched until `collect()`:

```
library(dbplyr)
diamonds_db <- tbl(con, "diamonds")

# Build query (lazy --- nothing executes yet)
big <- diamonds_db |>
  filter(price > 15000) |>
  select(carat:clarity, price)

big |> show_query()    # see generated SQL
big |> collect()       # execute and return tibble
```

`show_query()` is invaluable for learning SQL.

dplyr	SQL
<code>select()</code>	SELECT
<code>filter()</code>	WHERE
<code>arrange()</code>	ORDER BY
<code>group_by() + summarize()</code>	GROUP BY
<code>mutate()</code>	SELECT expr AS col
<code>rename()</code>	AS
<code>left_join()</code>	LEFT JOIN ... ON

```
flights |>
  filter(dest %in% c("IAH", "HOU")) |>
  show_query()
# SELECT * FROM flights WHERE dest IN ('IAH', 'HOU')
```

- 1 What SQL does `distinct()` translate to? What about `head(n = 10)`?
- 2 Use dbplyr to recreate: find flights where `dep_delay < arr_delay`, and compute speed (`distance / air_time * 60`).

① `distinct()` → `SELECT DISTINCT`. `head(10)` → `LIMIT 10`.

②

```
# Flights where dep_delay < arr_delay
flights_db |>
  filter(dep_delay < arr_delay) |> collect()

# Speed calculation
flights_db |>
  mutate(speed = distance / air_time * 60) |>
  select(tailnum, speed) |> collect()
```

Arrow and Parquet

Why Parquet?

Feature	CSV	Parquet
Size	Large (text)	50–60% smaller
Types	Guessed on read	Embedded metadata
Reading	Must read entire file	Column-selective
Speed	Slow for big data	Very fast

Parquet is a **columnar** format — reads only the columns you need.

Reading and Writing Parquet

```
library(arrow)

# Read large CSV lazily
data <- open_dataset("checkouts.csv",
  format = "csv",
  col_types = schema(ISBN = string()))

# Convert to partitioned parquet
data |>
  group_by(CheckoutYear) |>
  write_dataset("checkouts_pq",
    format = "parquet")

# Query parquet (lazy until collect)
open_dataset("checkouts_pq") |>
  filter(CheckoutYear == 2021) |>
  group_by(CheckoutMonth) |>
  summarize(total = sum(Checkouts)) |>
  collect()
```

Partitioning by year means filtering by year only reads the relevant files.

Exercises (Arrow)

- 1 Find the most popular book each year.
- 2 How have checkouts of physical books vs. ebooks changed over the last 10 years?

❶

```
open_dataset("checkouts_pq") |>  
  filter(MaterialType == "BOOK") |>  
  group_by(CheckoutYear, Title) |>  
  summarize(n = sum(Checkouts)) |>  
  slice_max(n, n = 1) |> collect()
```

- ❷ Filter `MaterialType` `%in% c("BOOK", "EBOOK")`, group by year and type, sum checkouts, plot as line chart.

Rectangling

List-Columns and Nested Data

Tibbles can hold **list-columns** — columns where each cell is a list:

```
df <- tibble(  
  x = 1:2,  
  y = list(list(a = 11, b = 12),  
            list(a = 21, b = 22)))  
df
```

```
#> # A tibble: 2 x 2  
#>       x y  
#>   <int> <list>  
#> 1     1 1 <named list [2]>  
#> 2     2 2 <named list [2]>  
#> ... [output truncated]
```

Two key operations to flatten:

- `unnest_wider()`: named list → new **columns**
- `unnest_longer()`: unnamed list → new **rows**

unnest_wider() — Named Lists to Columns

```
df |> unnest_wider(y)
```

```
#> # A tibble: 2 x 3  
#>       x       a       b  
#>   <int> <dbl> <dbl>  
#> 1     1     11     12  
#> 2     2     21     22
```

Each **name** in the list becomes a column. Use when elements have consistent names.

For nested lists, chain multiple unnest calls:

```
data |>  
  unnest_wider(json) |>      # level 1  
  unnest_wider(owner, names_sep = "_") # level 2
```

names_sep prevents column name conflicts.

unnest_longer() — Unnamed Lists to Rows

```
df2 <- tibble(  
  x = 1:3,  
  y = list(c(11, 12, 13), 21, c(31, 32)))  
df2 |> unnest_longer(y)
```

```
#> # A tibble: 6 x 2  
#>       x     y  
#>   <int> <dbl>  
#> 1     1    11  
#> 2     1    12  
#> 3     1    13  
#> 4     2    21  
#> 5     3    31  
#> ... [output truncated]
```

Each **element** becomes a row. The parent row is duplicated.

Working with JSON

JSON maps to R as: objects → named lists, arrays → unnamed lists.

```
json <- '[
  {"name": "John", "age": 34},
  {"name": "Susan", "age": 27}]'
```

```
tibble(json = jsonlite::parse_json(json)) |>
  unnest_wider(json)
```

```
#> # A tibble: 2 x 2
#>   name    age
#>   <chr> <int>
#> 1 John     34
#> 2 Susan     27
```

Strategy: start with `unnest_wider()`, inspect, then `unnest_longer()` for arrays, repeat.

Exercises (Rectangling)

- ❶ What happens if you use `unnest_wider()` on an unnamed list-column?
- ❷ What happens if you use `unnest_longer()` on a named list-column?
- ❸ Explain the difference between `hoist()` and repeated `unnest_wider()` calls.

Solutions (Rectangling)

- ❶ It errors — names are needed for column names. Use `names_repair` or provide names manually.
- ❷ It creates rows from the values and adds an extra column with the names (the index column).
- ❸ `hoist()` extracts deeply nested components in one step using path specifications (e.g., `c("bounds", "northeast", "lat")`). Repeated `unnest_wider()` flattens one level at a time. `hoist()` is more concise for deep extraction.

Web Scraping

```
library(rvest)
```

HTML is a tree of **elements**: `<tag attribute="value">content</tag>`

Tag	Purpose
<code><h1> – <h6></code>	Headings
<code><p></code>	Paragraphs
<code></code>	Links
<code><table></code>	Tables
<code>, </code>	Lists
<code><div>, </code>	Containers

Attributes: **id** (unique), **class** (group), **href** (link target).

Extracting Data with rvest

```
html <- read_html("https://example.com")

# Find elements with CSS selectors
html |> html_elements("h2")           # by tag
html |> html_elements(".classname")   # by class
html |> html_elements("#myid")        # by id

# Extract content
html |> html_elements("h2") |> html_text2()
html |> html_elements("a") |>
  html_attr("href")

# Tables → tibbles automatically
html |> html_element("table") |> html_table()
```

Nesting Pattern for Complex Pages

Critical: use `html_element()` (singular) inside a pipeline to maintain row alignment:

```
# Each <section> = one film
sections <- html |> html_elements("section")

tibble(
  title = sections |>
    html_element("h2") |> html_text2(),
  director = sections |>
    html_element(".director") |> html_text2()
)
```

`html_element()` returns `NA` for missing matches (keeps alignment). `html_elements()` drops non-matches (breaks alignment).

- **Public + non-personal + factual** = generally okay
- Always check **Terms of Service**
- Be careful with **personally identifiable information**
- Facts are not copyrighted; original content is
- Use the `polite` package to rate-limit requests
- `rvest` does **not** execute JavaScript — use `chromote` for dynamic sites